Collins Aerospace
An RTX Business

WiCI

Integrated Terrestrial and Satellite Communications

Final Report: Link-State Forecasting Audit, Distributional Models, and Calibrated Uncertainty

Jalal Jalali Evan Gossling Daji Qiao Hongwei Zhang
Iowa State University ▪ Center for Wireless, Communities and Innovation (WiCI)
Collins Aerospace: Naveen Rajanikantha, Ben Haan, Andrew Bys

Outline

1. Why forecast link state
2. Where the earlier work left us
3. Audit of the initial pipeline
4. What we changed (4 steps, with code)
5. How we evaluated
6. Results: accuracy and uncertainty
7. Which change matters, and how sure we are
8. Sensitivity studies
9. What it means for the scheduler
10. Limitations, next steps, deliverables

One-line summary

Against the initial model, on all six links: calibrated 95% interval score **-22.0%** and RTT MAE **-7.5%** on average, RMSE about unchanged.

Why forecast link state?

- A terminal with **Starlink, OneWeb, and 5G** paths can send each packet on the best one.
- The earlier **liquid tunnel** (MPQUIC) does this, but schedulers react to *past* RTT.
- LEO paths change fast:
  - Starlink reallocates users every **15 s**
  - OneWeb RTT jumps at every handover
- The earlier phase showed that forecasts help: prediction-aware minRTT cut frame loss per second by 52%.

What the scheduler needs

- RTT, jitter, loss forecasts **0.2 s to 3 s** ahead, every 200 ms
- **How wrong** each forecast may be (a missed deadline is costly)

Point accuracy is not enough. The scheduler needs trustworthy uncertainty.

Where the earlier work left us

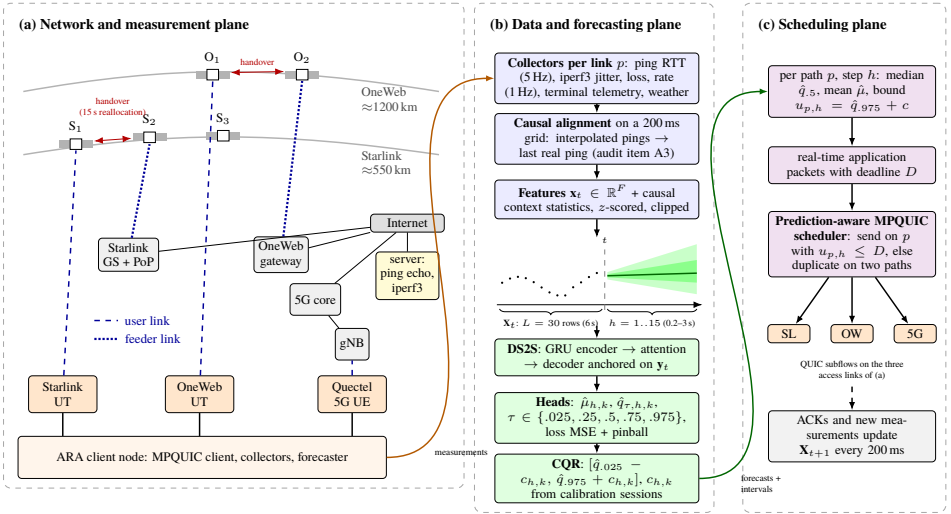
- 8 models compared on 6 links (Starlink, OneWeb, 5G $\times$ DL/UL)
- **Seq2Seq-GRU** with attention selected as the primary model
- Targets: RTT, jitter, loss
- Input 30 rows, horizon 15 steps
- Reported $\text{PICP}_{95} \approx 0.95$ everywhere "well-calibrated intervals"

Open questions for the final report

1. Are the numbers right? (data pipeline, metrics)
2. Can the forecasts be better?
3. Can the uncertainty be trusted on sessions the model never saw?

Before improving the model, we rebuilt and audited the evaluation.

System model



Data: six links measured on the ARA living lab

Link	Sessions (tr/va/te)	Train windows	Test windows	Features (Initial/Final)
OneWeb DL	45 / 9 / 11	117,558	27,492	9 / 31
OneWeb UL	45 / 9 / 11	115,847	28,125	9 / 31
Starlink DL	37 / 7 / 9	105,906	25,838	19 / 41
Starlink UL	37 / 7 / 9	106,165	25,666	19 / 41
5G DL	266 / 57 / 58	194,725	169,204	10 / 32
5G UL	265 / 56 / 58	193,502	168,519	10 / 32

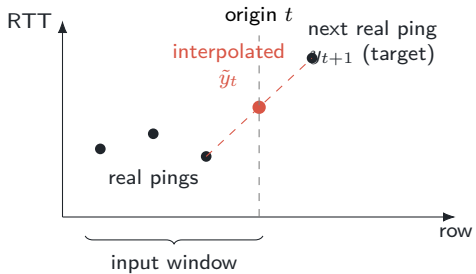
- One row every **200 ms**: input 30 rows = **6 s**, horizon 15 steps = **0.2–3 s**
- Split by **session** (70/15/15, seed 42), same as the earlier work: test sessions never seen
- 5G: every 4th training window used (7× more data), for all methods

Audit: six issues in the initial pipeline

Issue	Effect	Fix in the final report
A1 Horizon unit	"t+15s" is really **3 s** (200 ms steps)	report $h \times 0.2$ s
A2 Interval coverage	built from **test** residuals: 0.95 by construction	calibrate on validation sessions
A3 Starlink interpolation	**next ping leaks into the input**	last real ping, all methods
A4 Constant targets	OneWeb UL jitter/loss always 0, $R^2 = 1$ in aggregate	per-target, R^2 undefined
A5 "Relative RMSE"	not a persistence forecast	proper skill vs. $\hat{y}_{t+h} = y_t$
A6 One seed	report and stored run differ	3 seeds + ensembles

- Reproduced the earlier results first: Starlink DL MAE 8.36 vs. 8.55, RMSE 14.84 vs. 14.95
- 3 independent reviewers + 2 skeptics per finding, only surviving findings kept

A3: interpolation leaks the future into the input



Proposition (paper): with $\tilde{y}_t = \frac{1}{2}(y_{t-1} + y_{t+1})$,

$$y_{t+1} - \tilde{y}_t = \frac{1}{2}\,(y_{t+1} - y_{t-1})$$

The persistence error is **halved**, and the squared error is **4× smaller**, on every interpolated row.

- 11.9% of Starlink DL test targets interpolated
- fix raises 1-step persistence RMSE by **9.7%** (56.7 → 62.2 ms)
- every interpolated origin lies strictly between its neighbors (100%, vs. 47% for real pings)

All final results use causal inputs, for every method.

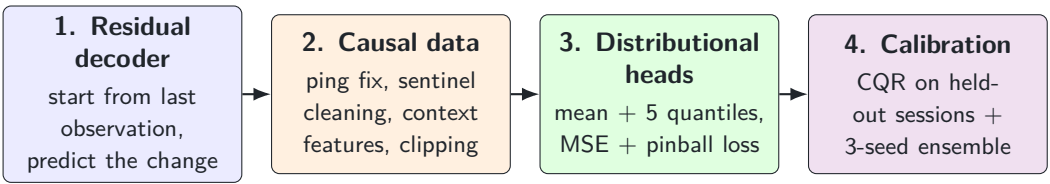
The fix in code (`dataset.py`)

```python
def interpolated_mask(values):
    # Real pings have at most 3 decimals; time-interpolated values do not.
    v = pd.to_numeric(values, errors="coerce")
    return ((v * 1000 - (v * 1000).round()).abs() > 1e-3) & v.notna()

def causal_ping_fix(df, provider):
    """Keep the measured series as the TARGET, make the INPUT causal."""
    for col in INTERPOLATED_PING_COLS.get(provider, ()):
        v = pd.to_numeric(df[col], errors="coerce")
        m = interpolated_mask(v)
        df[col + "__orig"]   = v                    # used as target
        df[col + "__interp"] = m.astype("float32")  # for analysis
        df[col] = v.mask(m).ffill()                 # input = last real ping
```

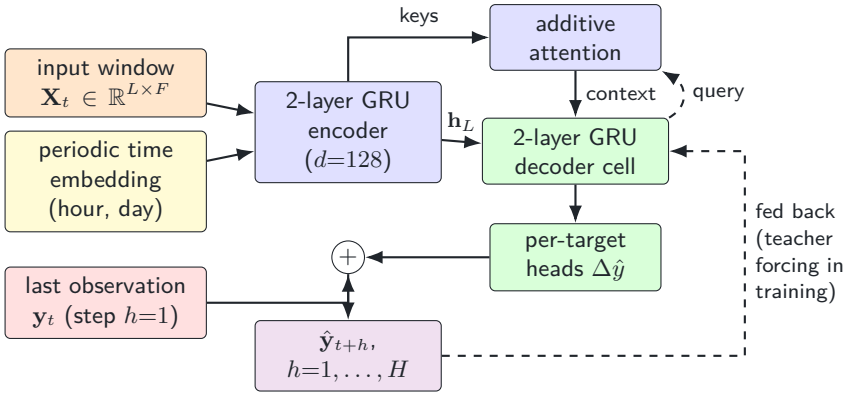
- Behind a flag: `DataConfig(causal_ping=True)`, so the earlier results can still be reproduced exactly
- Applied to **every** method in the study, including the retrained initial model

What we changed: four steps



- Encoder and attention are **unchanged** from the initial model, so the comparison isolates the decoder and the output
- Each step is evaluated separately (ablation, slide 23)
- New model: `models/seq2seq_dist.py`, 468,511 parameters (initial model: 449,592)

Final forecaster



Persistence is the forecast when every head outputs zero. The network learns the departure from it.

Steps 1–2: anchored decoder, context features

```python
# decoder step (seq2seq_dist.py)
delta = torch.stack([head(top)
                     for head in self.heads], 1)
out   = last.unsqueeze(-1) + delta   # anchored
last  = out[..., 0]                  # feed back mean
```

```python
# causal context features (dataset.py)
new[f"{c}__rsc"]    = rows_since_change(x)  # 0..10
new[f"{c}__mean5"]  = x.rolling(5).mean()    # 1 s
new[f"{c}__mean150"]= x.rolling(150).mean()  # 30 s
new[f"{c}__std300"] = x.rolling(300).std()   # 60 s
new[f"{c}__xmean"]  = x.expanding().mean()
```

Why

- The initial decoder started from **zero**, had to re-learn the level every time
- 6 s window is short: rolling 30–60 s statistics give longer context for free
- jitter/loss are 1 Hz reports held 5 rows: "rows since change" gives the phase
- 5G logs contain SINR $= -32768$ sentinels: set to missing, clip $|z| \leq 5$

Step 3: why a distribution instead of one number

- RTT at 5 Hz is **heavy tailed**: RMSE is 1.5–3.8× the MAE (1.25× for Gaussian errors)
- A squared-error model learns the **mean**, which spikes pull upward
- Each loss has its own best forecast (Gneiting 2011):
  - squared error → mean
  - absolute error → median
  - interval score → 2.5% and 97.5% quantiles

Design

Each head outputs $\hat{\mu}$ and $\hat{q}_\tau$, $\tau \in \{.025, .25, .5, .75, .975\}$

Loss: $(\hat{\mu} - y)^2 + \frac{1}{5}\sum_\tau \rho_\tau(y - \hat{q}_\tau)$

Report $\hat{\mu}$ for RMSE, $\hat{q}_{.5}$ for MAE, $[\hat{q}_{.025}, \hat{q}_{.975}]$ for intervals

One forward pass gives the right forecast for every use.

Step 3 in code: quantile heads and loss

```python
class DistHead(nn.Module):                # models/seq2seq_dist.py
    def forward(self, h):
        o = self.net(h)                    # 6 numbers per target and step
        mean, med = o[:, 0], o[:, 1]
        g = F.softplus(o[:, 2:]) + 1e-4  # positive gaps -> quantiles cannot cross
        q25, q75 = med - g[:, 0], med + g[:, 2]
        q025, q975 = q25 - g[:, 1], q75 + g[:, 3]
        return torch.stack([mean, q025, q25, med, q75, q975], dim=-1)

def dist_loss(out, y, quantile_weight=1.0):
    mse = ((out[..., 0] - y) ** 2).mean()                    # -> conditional mean
    tau = torch.tensor([0.025, 0.25, 0.5, 0.75, 0.975])
    e = y.unsqueeze(-1) - out[..., 1:]
    pinball = torch.maximum(tau * e, (tau - 1) * e).mean()  # -> conditional quantiles
    return mse + quantile_weight * pinball
```

Extra cost: 4% more parameters, 10.0 ms vs. 6.9 ms per forecast on one CPU thread (budget: 200 ms).

Step 4: calibrate on unseen sessions

Conformalized quantile regression (CQR)

- score on validation windows: $s_i = \max\{\hat{q}_{.025} - y_i,\ y_i - \hat{q}_{.975}\}$
- $c =$ the $\lceil (n+1)(0.95) \rceil$-th smallest score
- interval $[\hat{q}_{.025} - c,\ \hat{q}_{.975} + c]$, one c per target and step
- Initial model: same, around its mean $(\hat{y} \pm c)$
- 3-seed ensemble: average mean and every quantile

```
s = np.maximum(q_lo_val - y_val,
               y_val - q_hi_val)
k = np.ceil((n + 1) * (1 - alpha)) / n
c = np.quantile(s, k, method="higher")
lo, hi = q_lo_test - c, q_hi_test + c
```

Deployment: offsets $c_{h,k}$ saved in `summary/cqr_offsets_<link>.json`

Coverage is measured on test sessions, never computed from the test residuals.

How we evaluated

Methods (same split, same causal data)

- Persistence, 1 s seasonal repeat
- Ridge and Ridge-E (tuned linear)
- **Initial** Seq2Seq-GRU, retrained
- Ablation: +residual, +data
- **Final** distributional + ensemble

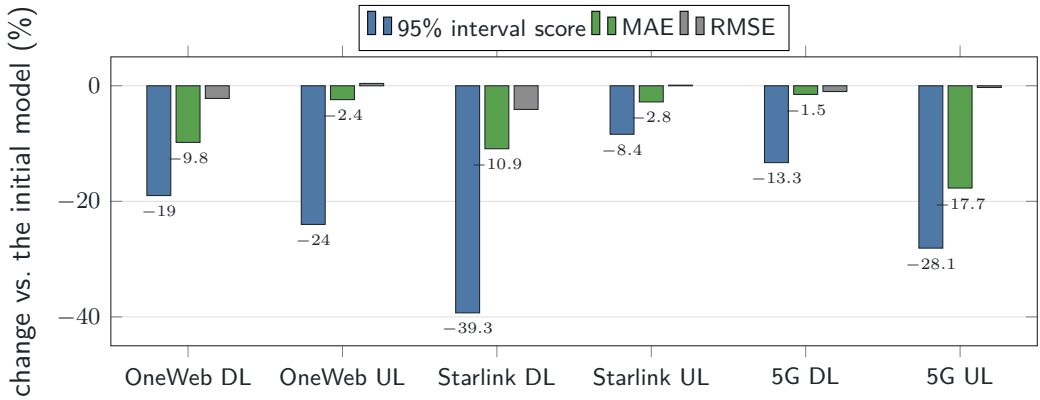
Protocol

- 3 seeds per neural model, 72 runs + 80 sweep runs
- Original optimizer settings, early stopping (stopped after 16–44 epochs)
- Apple M3 Max GPU, about 7 h

Metrics (test sessions)

- MAE, RMSE, R^2 per target and step
- skill vs. persistence
- **interval score** (MIS): width + $\frac{2}{\alpha}\times$ misses, lower is better
- coverage (PICP) of the calibrated 95% interval
- per-session Wilcoxon test

Headline: final vs. initial model on all six links (RTT)



Seed means (3 seeds each). Coverage of the final model's 95% intervals on test sessions: 0.935–0.954.

RTT results in detail: OneWeb and Starlink DL

Link	Model	MAE (ms)	RMSE (ms)	R^2	MIS (ms)	PICP
OneWeb DL	Initial (retrained)	17.99 ± 1.50	68.93 ± 1.26	0.175 ± 0.028	329.4 ± 5.4	0.953 ± 0.001
	Final	16.22 ± 0.03	67.38 ± 0.50	0.212 ± 0.013	266.7 ± 3.6	0.949 ± 0.006
	Initial ensemble	17.75	68.16	0.192	323.4	0.953
	Final ensemble	15.94	66.61	0.230	259.0	0.949
OneWeb UL	Initial (retrained)	71.83 ± 0.88	150.16 ± 0.86	0.746 ± 0.003	1059.8 ± 8.7	0.952 ± 0.000
	Final	70.10 ± 1.66	150.82 ± 5.08	0.743 ± 0.017	804.9 ± 19.4	0.950 ± 0.001
	Initial ensemble	68.76	145.43	0.762	1026.7	0.951
	Final ensemble	67.51	145.35	0.761	756.9	0.950
Starlink DL	Initial (retrained)	22.32 ± 0.57	37.72 ± 0.29	0.632 ± 0.006	246.7 ± 3.1	0.950 ± 0.000
	Final	19.89 ± 0.58	36.19 ± 0.17	0.661 ± 0.003	149.6 ± 1.3	0.954 ± 0.002
	Initial ensemble	21.60	37.04	0.645	243.5	0.950
	Final ensemble	18.45	35.20	0.679	143.4	0.953

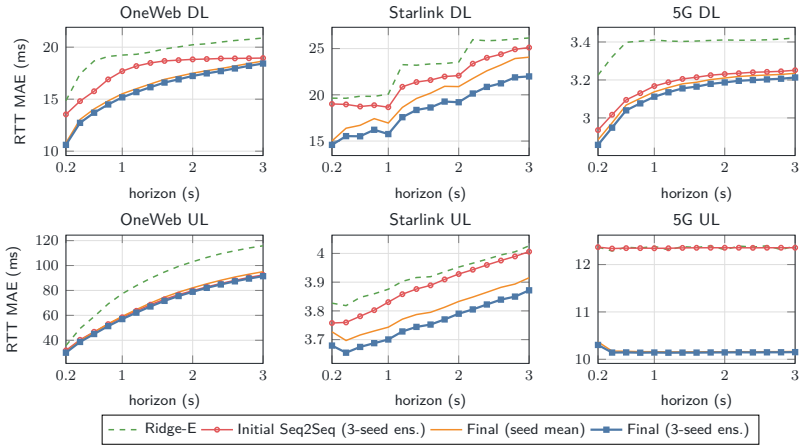
Mean $\pm$ std over 3 seeds, and 3-seed ensembles. MIS: calibrated 95% interval score. PICP: its measured coverage.

RTT results in detail: Starlink UL and 5G

Link	Model	MAE (ms)	RMSE (ms)	R^2	MIS (ms)	PICP
Starlink UL	Initial (retrained)	3.91 ± 0.07	7.07 ± 0.03	0.351 ± 0.005	38.4 ± 0.1	0.932 ± 0.001
	Final	3.80 ± 0.02	7.08 ± 0.03	0.350 ± 0.005	35.2 ± 1.7	0.935 ± 0.009
	Initial ensemble	3.88	7.04	0.357	38.2	0.930
	Final ensemble	3.76	7.04	0.358	34.6	0.936
5G DL	Initial (retrained)	3.20 ± 0.00	4.64 ± 0.01	0.231 ± 0.004	25.3 ± 0.2	0.953 ± 0.001
	Final	3.15 ± 0.01	4.59 ± 0.01	0.246 ± 0.002	22.0 ± 0.0	0.950 ± 0.001
	Initial ensemble	3.18	4.61	0.239	25.2	0.953
	Final ensemble	3.12	4.57	0.254	21.7	0.950
5G UL	Initial (retrained)	12.36 ± 0.08	19.90 ± 0.00	0.016 ± 0.000	127.9 ± 0.5	0.948 ± 0.000
	Final	10.18 ± 0.01	19.84 ± 0.00	0.022 ± 0.000	92.0 ± 0.1	0.949 ± 0.001
	Initial ensemble	12.35	19.89	0.017	127.9	0.948
	Final ensemble	10.15	19.83	0.023	91.7	0.950

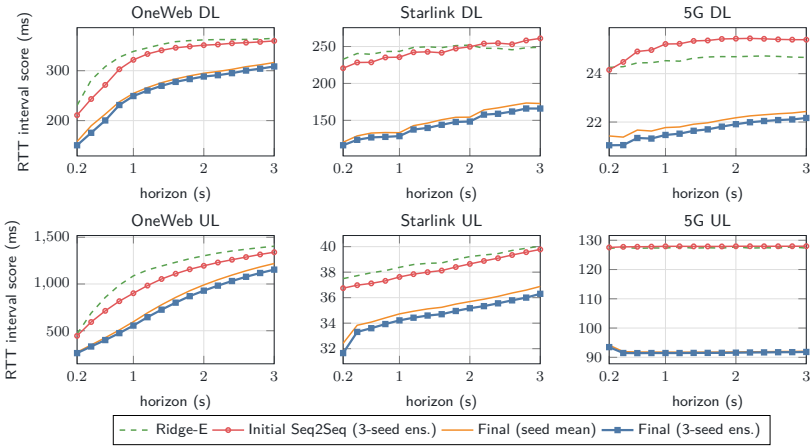
Mean $\pm$ std over 3 seeds, and 3-seed ensembles. MIS: calibrated 95% interval score. PICP: its measured coverage.

Per horizon step: RTT MAE



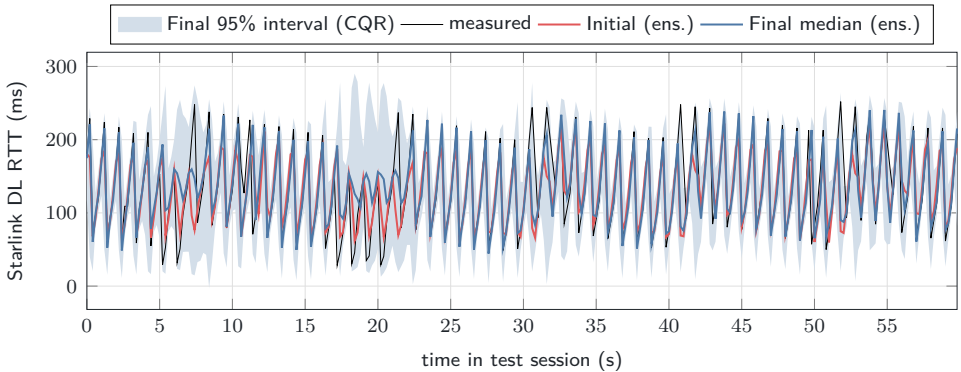
Final model below initial model at every step (0.2–3 s) on every link.

Per horizon step: calibrated 95% interval score



Largest gains on Starlink DL, OneWeb, and 5G UL, where RTT is bursty.

Why the intervals are better: they follow the input



- Final interval widens around bursts, narrows when calm. Width–error rank correlation 0.17–0.64 on all links
- Starlink DL 95% interval: **100 ms** wide (Final) vs. **176 ms** (Initial), same coverage

Which step helps? (ablation vs. initial model)

Steps	OneWeb DL			OneWeb UL			Starlink DL			Starlink UL			5G DL			5G UL		
	MAE	RMSE	MIS	MAE	RMSE	MIS	MAE	RMSE	MIS	MAE	RMSE	MIS	MAE	RMSE	MIS	MAE	RMSE	MIS
1: residual decoder	+9.8	-0.9	-1.1	+2.7	+3.3	+4.5	+0.7	-0.1	-1.1	+0.2	+0.2	+0.7	+0.9	+0.2	-0.7	+0.7	+0.1	-0.1
1+2: + data pipeline	+19.0	-2.8	-1.5	-1.2	-2.5	-2.6	-7.8	-4.5	-1.9	-0.7	+0.1	+0.9	+0.6	-0.6	-2.4	+0.1	-0.3	-0.5
1+2+3: + distrib. heads	-9.8	-2.2	-19.0	-2.4	+0.4	-24.0	-10.9	-4.1	-39.3	-2.8	+0.1	-8.4	-1.5	-1.0	-13.3	-17.7	-0.3	-28.1
1–4: + 3-seed ensemble	-11.4	-3.4	-21.4	-6.0	-3.2	-28.6	-17.3	-6.7	-41.9	-3.9	-0.5	-9.8	-2.2	-1.6	-14.4	-17.8	-0.4	-28.3

- Steps 1–2 alone do **not** lower the MAE (OneWeb DL $+9.8\%$, $+19\%$): a squared-error model anchored on a spiky value follows the spikes
- Step 2 helps the **mean**: up to -4.5% RMSE (Starlink DL)
- **The gains come from Step 3**, the distributional output. Ensembling adds up to 6 points

The evaluation method hides or shows the gain

	Earlier method			Calibrated		
Link	Initial	Final	Δ%	Initial	Final	Δ%
OneWeb DL	309.1	315.6	+2.1	329.4	266.7	-19.0
OneWeb UL	1043.6	1039.8	-0.4	1059.8	804.9	-24.0
Starlink DL	245.9	243.5	-1.0	246.7	149.6	-39.3
Starlink UL	36.8	37.3	+1.2	38.4	35.2	-8.4
5G DL	24.7	24.4	-1.5	25.3	22.0	-13.3
5G UL	97.3	97.5	+0.2	127.9	92.0	-28.1

RTT interval score (ms)

- **Earlier method**: quantiles of test residuals around the mean → both models within $\pm 2\%$
- **Calibrated** on held-out sessions → final model 8–39% better

Test-residual intervals hide the value of input-dependent uncertainty and overstate coverage.

How sure are we? Per-session significance

Link	Sessions	Final wins	Median Δ%	p
OneWeb DL	11	11	-9.3	0.0010
OneWeb UL	11	8	-2.2	0.1475
Starlink DL	9	8	-15.6	0.0078
Starlink UL	9	9	-2.6	0.0039
5G DL	58	48	-1.6	$< 10^{-4}$
5G UL	58	58	-18.5	$< 10^{-4}$

- Windows of one session are correlated, so the **session** is the unit of evidence
- The final model has lower RTT MAE in **142 of 156** test sessions
- Wilcoxon $p < 0.01$ on 5 of 6 links
- OneWeb UL: 8 of 11, not significant ($p = 0.1475$)

Honest comparison with simple baselines

Method	OneWeb DL MAE	OneWeb DL skill	OneWeb UL MAE	OneWeb UL skill	Starlink DL MAE	Starlink DL skill	Starlink UL MAE	Starlink UL skill	5G DL MAE	5G DL skill	5G UL MAE	5G UL skill
Persistence	21.23	0.0	89.14	0.0	44.56	0.0	5.16	0.0	4.90	0.0	15.83	0.0
Seasonal repeat (1 s)	22.59	-8.7	101.0	-12.4	21.70	28.3	5.18	-0.9	5.03	-1.8	15.93	-0.2
Ridge	20.96	21.3	98.38	8.6	23.43	40.5	3.98	25.1	3.42	28.6	12.59	-43.3
Ridge-E	19.42	22.5	87.93	13.0	23.04	41.7	3.92	25.9	3.39	29.5	12.36	29.1
Initial Seq2Seq-GRU	17.99	22.7	71.83	20.2	22.32	41.4	3.91	26.3	3.20	30.8	12.36	28.8
Final	16.22	24.5	70.10	19.8	19.89	43.8	3.80	26.2	3.15	31.5	10.18	29.1
Final ensemble	15.94	25.3	67.51	22.8	18.45	45.3	3.76	26.6	3.12	31.9	10.15	29.1

- Persistence is poor at 5 Hz (single pings are noisy). Starlink DL has a 1 s pattern (seasonal repeat: 28% skill)
- A tuned linear model (Ridge-E) is **within 1 point** of the initial network's RMSE skill on 4 links
- Final ensemble: **lowest MAE and highest skill on every link** (tied on 5G UL)

Sensitivity studies (80 extra runs, satellite links)

Setting (Final vs. Initial, %)	OneWeb DL MAE	OneWeb DL RMSE	OneWeb DL MIS	OneWeb UL MAE	OneWeb UL RMSE	OneWeb UL MIS	Starlink DL MAE	Starlink DL RMSE	Starlink DL MIS	Starlink UL MAE	Starlink UL RMSE	Starlink UL MIS
default (6 s input, 3 s horizon)	-13.7	-3.0	-18.6	-1.1	+2.1	-23.3	-11.8	-4.1	-39.0	-3.7	-0.3	-7.2
input 15 rows (3 s)	-5.4	-1.4	-18.9	-11.1	-7.8	-29.9	-10.0	-4.1	-41.1	-1.8	-0.6	-12.3
input 60 rows (12 s)	-3.6	-2.1	-19.0	-11.4	-6.2	-27.2	-10.3	-2.4	-37.2	-1.5	-1.3	-12.5
horizon 30 steps (6 s)	-6.3	-0.7	-10.4	-4.6	-1.4	-14.7	-0.2	-2.1	-35.6	-2.2	+0.2	-10.4

- The final model beats the initial one in **every** window and horizon setting: MIS 7–41% better, MAE lower everywhere
- **Pinball weight 4** (instead of 1) improves the final model further: OneWeb DL MAE -10.4%. Our main tables understate the model
- Best window depends on the link (12 s: OneWeb UL -9.4%, OneWeb DL $+11.2\%$). Hidden size 128 is adequate

What it means for the multipath scheduler

Per-path risk bound

- $u_{p,h} = \hat{q}_{.975} + c$ holds ≈95% of the time on unseen sessions
- send on path p if $u_{p,h} \leq$ deadline, else duplicate on two paths
- tight on calm paths ⇒ fewer needless duplicates

Spike warning (RTT > 90th pct., AUROC)

- Starlink DL: **0.901** (Final) vs. 0.832 (Initial)
- similar on 4 links, worse on OneWeb DL (0.732 vs. 0.765)

Which output for which decision

- **median** $\hat{q}_{.5}$: rank paths by expected delay
- **mean** $\hat{\mu}$: capacity, throughput estimates
- **upper bound**: deadline and duplication decisions

Real-time: 10.0 ms per forecast on one CPU thread, every 200 ms

Limitations and next steps

Limitations

- RTT **RMSE did not improve** by more than 7% anywhere
- Coverage measured, not guaranteed (sessions not exchangeable)
- One site, constant-rate iperf traffic, one split
- iperf reports stamped at interval start
- Scheduler benefit argued, not yet measured end to end

Next steps

- Run the final model inside the liquid tunnel (end-to-end video test)
- Pinball weight 4, per-link input window
- Cross-path features (state of the other links)
- Online (adaptive) conformal updates
- Fix iperf timestamps in the parsers

Summary and deliverables

- **Audit**: found and fixed a future-data leak in Starlink RTT and a coverage metric that was 0.95 by construction
- **Model**: same encoder, anchored decoder, **distributional heads**, CQR calibration, ensembles
- **Results**: interval score **8–39%** better, MAE **2–18%** lower, all six links, significant in 5 of 6

Deliverables (github.com/Jalalij/Collins_2026)

final report (53 pages) ▪ IEEE TWC paper draft (10 pages, 40 refs) ▪ code and benchmark scripts ▪ 202 trained models ▪ measurement data ▪ supplementary docs (changes, data audit, reproduction)

Questions?